

Usage

To use NCCL Miner, follow these steps:

1. Enable Tracing in Your Training Script

Use the PyTorch Profiler to annotate your training code. For example:

```
with torch.profiler.profile(...):  
    <your distributed training code>
```

2. Enable NCCL Debug Logging

Set the following environment variables before running your training:

```
export NCCL_DEBUG=TRACE  
export NCCL_DEBUG_FILE=nccl_log_%h_%p.txt
```

3. Run Your Training Script

Execute the training script as usual. Each GPU process will produce its own NCCL log file and a PyTorch profile trace.

4. Parse the Logs Using NCCL Miner

After training, run the NCCL Miner parser script:

```
python main.py <path_to_nccl_logs>  
<path_to_torch_profiler_output> -o <optional_output_name>
```

5. Visualize in Perfetto

Open the JSON file in ui.perfetto.dev to explore the full data and communication flow.

What's in JSON file

- Data Flow
 - o Each data flow is represented using two JSON objects: one for the **Send** and one for the **Receive**. Both include a start timestamp (ts) and duration (dur), in microseconds.
 - o Additionally, the JSON object also includes the source/destination GPU (pid) and the data size (bytes) of the data flow.
 - o Example:

```
{  
  
    "ph" :  "X" ,
```

```

    "cat": "data_flow",
    "name": "Send to 1",
    "pid": 0,
    "tid": 200,
    "ts": 3690841421320.032,
    "dur": 7.423828125,
    "args": {
        "bytes": 8,
        "name": "0's Data"
    },
    "id": 0
},
{
    "ph": "X",
    "cat": "data_flow",
    "name": "Recv from 0",
    "pid": 1,
    "tid": 200,
    "ts": 3690841421509.171,
    "dur": 1.0,
    "args": {
        "bytes": 8,
        "name": "0's Data"
    },
    "id": 0
},

```

- Flow Dependencies

- The dependencies are represented as “arrows” using a start event (“ph”: “s”) and a finish event (“ph”: “f”).
- Each event has an associated timestamp (ts). The timestamp of the start event represents the end of the previous data flow, and the timestamp of the finish event represents the start of the following data flow.
- Example:

```
{
  "ph": "s",
  "cat": "dependency",
  "id": 732,
  "pid": 1,
  "tid": 200,
  "ts": 3690841421510.171,
  "bind_id": 0
},
{
  "ph": "f",
  "cat": "dependency",
  "id": 732,
  "pid": 1,
  "tid": 200,
  "ts": 3690841421511.171,
  "bind_id": 1,
  "bp": "e"
},
```

Current Limitation

- The current output JSON file is made specific for Perfetto UI visualization. Personally, I think we should generate a separate JSON file designated for the simulator.

- It should be a simple change since the data flow and dependencies are already extracted.
- The duration of data flows are not accurately estimated.
 - The duration is too small for Collective operations.
 - Potential solution: estimate duration based on hardware info, such as bandwidth between GPUs
 - If the simulator contains the feature of estimating data flow durations, then this will not be an issue.